

Trusted AI Safety Expert: Module 5

Official Study Guide



Version Number: 20251015

© 2025 Cloud Security Alliance – All Rights Reserved. You may download, store, display on your computer, view, print, and link to the Cloud Security Alliance at <https://cloudsecurityalliance.org> subject to the following: (a) the draft may be used solely for your personal, informational, noncommercial use; (b) the draft may not be modified or altered in any way; (c) the draft may not be redistributed; and (d) the trademark, copyright or other notices may not be removed. You may quote portions of the draft as permitted by the Fair Use provisions of the United States Copyright Act, provided that you attribute the portions to the Cloud Security Alliance.

Acknowledgments

Lead Authors

Tugce Guler
Satwik Kamarthi
Wei Li
Charlie Meyer
Nihar Sanda
Sam Scarpino
James Sheldon
Thulasi Tholeti
Peter Vickers

Contributors

Cansu Canca
Shiran Dudy
Laura Haaber Ihle
Shantanu Jain
Tomo Lazovich
Resmi Ramachandranpillai
Annika Schoene

CSA and NU Global Staff

Marina Bregkou
Josh Buker
Daniele Catteddu
Holly Franklin
Ryan Gifford
Keagan Goodhue
Tara Hanson
Larry Hughes
Nicholas Knopf
Stephen Lumpe
NJ Park
Hannah Rock
Anna Schorr
Stephen Smith
Cathy Jo Swain
Kevin Tobin
Jamie Traynor
Kira Twombly

Table of Contents

- Acknowledgments.....3
 - Lead Authors.....3
 - Contributors.....3
 - CSA and NU Global Staff.....3
- Table of Contents.....4
- Introduction to the TAISE.....5
 - Receive a Respected Industry Credential.....5
 - Acquire In-Demand Skills.....5
 - Go Beyond Theory.....5
- Module 5: AI Model Lifecycle & Threat Taxonomy.....6**
 - Learning Objectives.....6
 - 5.2 Module Vocabulary.....7
 - 5.3 Phases of AI Model Development.....9
 - 5.4 Case Study: An AI Model Development Lifecycle.....14
 - 5.5 Understanding Threats.....20
 - 5.6 Introduction to Threat Modeling.....26
 - 5.7 Periodic Risk Reviews and Scenario-Based Risk Assessments.....30
 - 5.8 Risk for AI Models.....33
 - 5.9 Risk Management Using NIST AI RMF.....35
 - 5.10 ISO and IEC Standards for AI Risk Management.....39
 - 5.11 MITRE ATLAS: Adversarial Threat Landscape for AI Systems.....43
 - 5.12 Summary.....47
- Change Log.....49

Introduction to the TAISE

If AI is going to shape the world, we need people who know how to shape it safely.

The **Trusted AI Safety Expert (TAISE)** certificate, created by the Cloud Security Alliance (CSA) and Northeastern University, is a rigorous, research-backed program for professionals who build, manage, or audit intelligent systems. Through 10 comprehensive modules and a final certificate exam, learners gain practical skills to evaluate and mitigate real-world AI risks, apply AI safety and security controls, navigate compliance frameworks, and lead responsible AI adoption across industries. TAISE is more than a certificate—it's a commitment to advancing safe, secure, and responsible AI.

Receive a Respected Industry Credential

TAISE is co-issued by CSA, **the global leader in cloud security standards**, and Northeastern University, **a top academic institution for AI ethics and experiential learning**. This joint recognition enhances your professional credibility, providing a globally respected credential to highlight on resumes, LinkedIn, and professional profiles.

Acquire In-Demand Skills

Professionals will learn to work with frameworks such as **NIST AI RMF, CSA AICM, ISO standards**, and **MITRE A TLAS** while mastering **threat classification, data privacy, model governance, and AI-specific incident response**. These capabilities directly map to the urgent needs of enterprises adopting AI.

Go Beyond Theory

The **TAISE Prompt Library** transforms concepts into hands-on practice. It offers curated prompts and exercises that show learners how to use large language models to reinforce and apply safety principles from each module. While not an exam prep tool, it helps professionals **translate training into job-ready skills**.



Module 5: AI Model Lifecycle & Threat Taxonomy

In this module, you will develop a deep understanding of both the technical journey of AI systems—from conception to deployment—and the critical security challenges that emerge at every stage. You'll begin by diving deeper into the complete AI model development life cycle, examining the development phases from initial problem definition through data collection, preprocessing, and model training, all the way to deployment, monitoring, and governance. This foundational knowledge will provide you with the context needed to understand where vulnerabilities can emerge and how different stages of development create unique risk profiles.

Building on this understanding, the module will immerse you in the complex landscape of AI-specific threats. You'll learn to identify and analyze sophisticated attack vectors, including data poisoning, model manipulation, evasion attacks, and privacy-threatening techniques such as membership inference attacks. The module also discusses the hands-on application of industry-standard risk assessment frameworks—including NIST AI RMF, ISO/IEC standards, and MITRE ATLAS—equipping you with practical tools for threat modeling, scenario planning, and implementation of continuous risk management practices. By the end of this module, you'll be prepared to conduct security assessments of AI systems and develop strategies for protecting these critical technologies throughout their operational life cycle.



Access the [TAISE Prompt Library](#), a free collection of curated prompts designed to complement the TAISE training. This optional resource provides example prompts and techniques that show how to use large language models (LLMs) to explore, reinforce, and apply key ideas from each module.

Learning Objectives

In this module, you will learn to:

- Outline the phases of the AI model life cycle—from preparation to monitoring
- Classify potential threats to AI models (e.g., data poisoning, model manipulation, sensitive data disclosure)
- Implement risk assessment strategies and scenario planning using frameworks such as NIST AI RMF, ISO standards, and MITRE ATLAS
- Explore how periodic risk reviews support ongoing safety management

5.2 Module Vocabulary

An **activation function** is a mathematical function applied to the output of each neuron (node) in a neural network that determines whether and how strongly that neuron should be "activated." It introduces nonlinearity into the network, which is crucial for the network's ability to learn complex patterns.

A **backdoor attack** occurs when an attacker modifies training data by inserting a specific trigger pattern into multiple samples and mislabeling them, causing the trained model to behave maliciously when the trigger is present while functioning normally otherwise.

Clean-label poisoning is a sophisticated data poisoning technique where attackers modify legitimate samples to mimic features of a different class while keeping their original labels, corrupting the model's decision boundaries without obvious label inconsistencies.

Concept drift occurs when the relationship between input features and target outputs changes over time, causing model performance to degrade even when input distributions remain stable.

Data drift refers to changes in the statistical properties or distribution of input data over time, which can cause the model's performance to degrade if the new data differs significantly from the training distribution.

In **data poisoning**, an attacker introduces malicious data into a model's training dataset. This corrupts it and causes it to learn biased or incorrect information, so that when users submit queries, the responses the users receive become lower in quality. An attacker may introduce bad data directly, through insider or backdoor access, or through the model's supply chain, meaning its dependencies such as third-party software libraries. A compromised model may show lower reliability and accuracy, become more vulnerable to future attacks, and share confidential information.

An **evasion attack** occurs when an attacker crafts inputs specifically designed to mislead a trained model at inference time, exploiting the model's sensitivity to small input variations and causing misclassification.

A **logic bomb** is malicious logic embedded within an AI model that activates only under specific predefined conditions, causing the model to behave maliciously when triggered while operating normally under typical circumstances.

A **loss function** is a mathematical function that measures how far off a machine learning model's predictions are from the actual correct answers (called ground truth or target values). It quantifies the "cost" or "penalty" of making incorrect predictions.

A **membership inference attack** occurs when an adversary attempts to determine whether a specific data point was part of a model's training set by analyzing the model's output patterns, potentially revealing sensitive information about individuals.

Model extraction is an attack aimed at stealing or replicating a trained machine learning model by querying it with various inputs and observing outputs to train a surrogate model with similar functionality.

Model manipulation occurs when an adversary gains access to a machine learning model and subtly

alters it to change its behavior in malicious ways, often through backdoor injection, weight perturbation, or logic bomb insertion.

Multicollinearity occurs when two or more predictor variables (independent variables) in a regression model are highly correlated. This means that one predictor variable can be linearly predicted from the others with a substantial degree of accuracy. When multicollinearity is present, it becomes difficult to determine the individual effect of each correlated variable on the dependent variable, since they tend to change together.

High collinearity refers to an extreme case of multicollinearity where the correlation between predictor variables is very strong (typically above 0.8 or 0.9). In cases of perfect collinearity, one variable can be perfectly predicted from another, making the regression model mathematically impossible to estimate.

Opaque decision boundaries refer to the complex, nonlinear way a model makes decisions, making it difficult to understand or explain its reasoning.

Repudiation in the context of AI means an inability to audit or trace an action back to a specific user or input due to insufficient logging or the opaque nature of the model.

An organization's **risk appetite** refers to the amount and type of risk an organization is willing to accept in pursuit of its objectives. For AI systems, this includes tolerance for model errors, bias levels, security vulnerabilities, and compliance gaps.

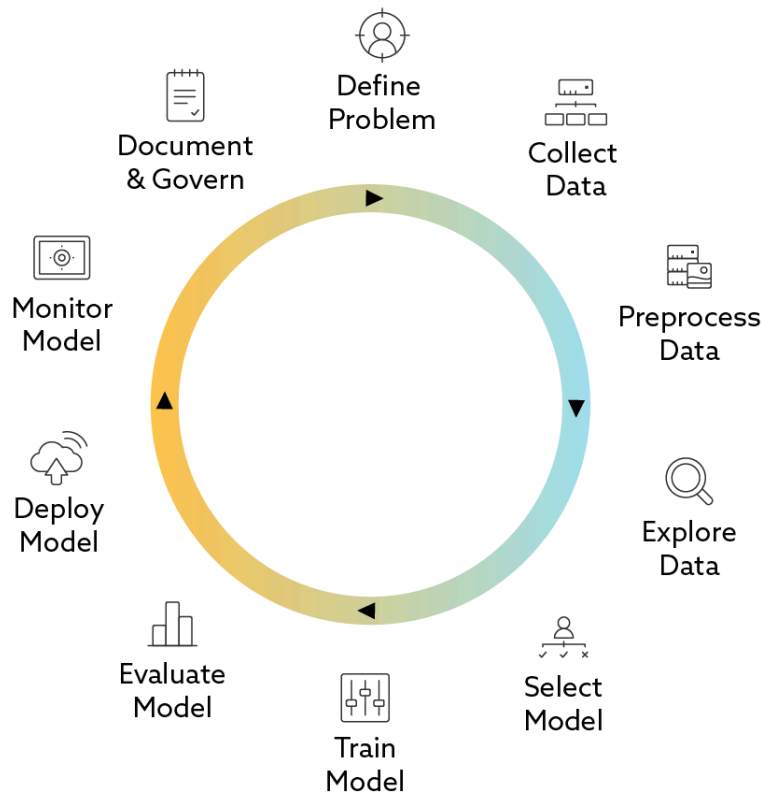
TEVV (Test, Evaluation, Verification, Validation) is a rigorous methodology for implementing systematic testing throughout the AI life cycle to ensure systems meet specified requirements and behave as intended.

Threat modeling is a systematic, proactive approach to identifying, evaluating, and addressing potential security vulnerabilities in a system by analyzing attack vectors, threat actors, and defensive controls.

5.3 Phases of AI Model Development

A Closer Look: The AI Model Development Life Cycle

The image below outlines the phases of AI model development. You can click on the elements below the image for explanations of each of the phases.



<Alternative text: The stages of the AI model development life cycle are: Define problem, collect data, preprocess data, explore data, select model, train model, evaluate model, deploy model, monitor model, and document and govern.>

Define Problem

The foundation of successful AI model development lies in articulating a clear business problem. This starts with the fundamental question of whether artificial intelligence is the appropriate solution. Often, business problems can be solved through simpler approaches such as statistical modeling or probability theory. Once AI is deemed suitable, the challenge becomes translating abstract business objectives into concrete machine learning tasks—such as classification (distinguishing between categories like dogs versus cats), regression (predicting continuous values), or clustering (grouping similar entities).

Comprehensive AI problem definition requires extensive stakeholder engagement to precisely specify desired outputs, understand available inputs and resources, and establish measurable success criteria. These success metrics typically include quantitative measures such as accuracy, F1 score, precision, or recall, depending on the specific use case and business priorities.

When defining the problem, an organization must consider whether an off-the-shelf or pre-trained model is sufficient or a custom-built solution is necessary. This phase should include a vendor and model analysis to determine the suitability of third-party options based on their capabilities, limitations, and security posture. This helps to determine whether fine-tuning or prompt engineering is a viable path forward.

Collect Data

With a well-defined problem in hand, the focus shifts to gathering the data necessary for model training. Modern data ecosystems offer diverse sources, ranging from structured databases and real-time APIs to IoT sensor streams, public repositories, research studies, and user-generated content. The critical challenge lies not in finding data but in identifying sources that align with problem requirements while maintaining quality and relevance. Source selection directly impacts model accuracy and generalization capabilities.

Given data's inherently dynamic nature, robust extraction and consolidation processes are essential for maintaining consistency. Throughout this phase, practitioners must navigate complex legal and ethical landscapes—ensuring compliance with regulations, obtaining necessary permissions, and adhering to IRB (Institutional Review Board) protocols when handling academic or research data. Key considerations include managing the “three Vs” (volume, variety, and velocity) of incoming data while addressing critical concerns around privacy protection and potential biases that could compromise model fairness.

Preprocess Data

After collecting data, the next critical phase involves cleaning and transforming it into a format suitable for machine learning algorithms. Raw data is rarely perfect—it often contains missing values, inconsistencies, and formatting issues that can significantly impact model performance. The preprocessing phase addresses these challenges through various data cleaning methods: handling missing values (through imputation or removal), normalizing numerical features to ensure similar scales, and encoding categorical variables into numerical representations that algorithms can understand.

This phase also involves the crucial step of splitting the dataset into training, validation, and test sets to ensure proper model evaluation. Tools like Pandas and NumPy in Python provide the foundation for these transformations.

When the dataset is insufficient or suffers from class imbalance, various data augmentation techniques can artificially increase the data size and balance.

Instead of collecting and preprocessing massive datasets for training from scratch, this phase shifts to collecting data for fine-tuning. This smaller, more specific dataset needs careful cleaning and labeling to avoid introducing new biases or vulnerabilities into the pre-trained model. Additionally, this stage should involve an analysis of the pre-trained model's original training data, if possible, to understand its potential biases or weaknesses.

Exploratory Data Analysis (EDA)

Before diving into model building, it's essential to thoroughly understand your data through exploratory analysis. EDA acts as detective work for machine learning, where practitioners uncover hidden patterns,

identify potential issues, and gain insights that inform modeling decisions. This phase involves creating various visualizations: histograms to understand distributions, scatterplots to identify relationships between variables, and correlation matrices to detect multicollinearity.

Statistical summaries help quantify what the visualizations reveal, while careful analysis can expose critical issues such as class imbalance, outliers, or high collinearity between features that could derail model performance. Tools such as Matplotlib, Seaborn, and Plotly transform raw numbers into compelling visual stories that guide the entire modeling process. The insights gained during EDA often lead to additional preprocessing steps or inform feature engineering decisions.

Class imbalance can be addressed using data augmentation techniques, while outliers can be handled through appropriate removal or transformation methods.

Select Model

With clean, well-understood data in hand, the focus shifts to choosing the right algorithm for the task. Model selection is both an art and a science, requiring careful consideration of the problem type, data characteristics, and project constraints. This phase involves experimenting with multiple algorithms—from simple linear models such as logistic regression to complex ensemble methods such as random forests or deep neural networks. Transfer learning techniques can leverage pre-trained models to reduce training time and computational requirements.

Each model comes with its own trade-offs: Simpler models offer better interpretability but may lack the capacity to capture complex patterns, while sophisticated models can achieve higher accuracy at the cost of being obscured or opaque. The key is finding the sweet spot between model complexity and performance that aligns with business requirements and computational resources. Incorporating explainable AI techniques can provide additional value. Understanding these trade-offs helps practitioners make informed decisions rather than defaulting to the most complex option.

The focus shifts at this critical new step from choosing a model type (e.g., neural network, random forest) to selecting a specific pre-trained model from a third-party provider. The life cycle needs to include a due diligence process to assess the model's security, its license terms, and the provider's security practices. This is also the phase where prompt engineering becomes a key activity, as it's the primary way to control a model's output without extensive retraining.

Train Model

Training transforms a chosen algorithm into a functional model by learning patterns from prepared data. This iterative process involves more than just feeding data into an algorithm—it requires careful feature selection to identify the most informative variables, hyperparameter tuning to optimize model configuration, and cross-validation to ensure robust performance estimates. For neural networks, this includes choosing appropriate loss functions, activation functions, batch sizes, and learning rates. An activation function receives the output from a node in a neural network, such as yes or no, 0 to 1, etc., depending on the function. The purpose of an activation function is to add nonlinearity to the neural network. A loss function is a method of evaluating how well your algorithm is modeling your dataset. It is used to optimize the model during training.

Techniques such as grid search and random search systematically explore different hyperparameter combinations to find optimal settings. The training phase is computationally intensive and often requires multiple iterations as practitioners experiment with different feature sets and parameter configurations. Modern frameworks such as Scikit-learn, TensorFlow, and PyTorch provide powerful tools for efficient training, but success still depends on thoughtful experimentation and domain expertise.

Evaluate Model

Once trained, a model must prove its worth through rigorous evaluation on unseen data. This critical phase determines whether the model genuinely learned useful patterns or merely memorized the training data. Evaluation involves applying the model to a held-out test set and measuring performance using problem-appropriate metrics—accuracy, precision, recall, and F1 score for classification tasks, or RMSE, MAE, and R^2 for regression problems.

Tools such as confusion matrices and precision-recall curves provide deeper insights into model behavior across different scenarios. Best practices include assessing overfitting and underfitting, analyzing the bias-variance trade-off, and performing A/B tests when applicable. The evaluation results often reveal the need to revisit earlier phases, creating an iterative cycle of improvement.

The evaluation process must now include testing for vulnerabilities specific to the third-party model, such as prompt injection and adversarial attacks. While traditional performance metrics are still important, the emphasis is on testing how the model responds to unexpected or malicious inputs that were not part of its original training. This phase also includes evaluating the fine-tuned model for any new biases that may have been introduced.

Deploy Model

A model that performs well in development means nothing if it can't deliver value in production. Deployment bridges the gap between experimental success and real-world impact, transforming a trained model into an accessible service. This phase involves packaging the model into a production-ready format—whether as a REST API for real-time predictions, a batch processing pipeline for periodic scoring, or an edge deployment for IoT devices.

The deployment strategy depends on factors such as latency requirements, expected traffic, and infrastructure constraints. Modern deployment tools such as Flask and FastAPI simplify API creation, while containerization with Docker and orchestration with Kubernetes ensure scalability and reliability. MLflow and similar platforms provide end-to-end solutions for model packaging, versioning, and deployment. Thorough integration testing in a staging environment is essential before production release.

Monitoring and Maintenance

In production, models face the reality of changing data distributions, evolving user behaviors, and shifting business contexts. Continuous monitoring tracks key performance indicators to detect degradation before it impacts business outcomes. This monitoring involves watching for data drift (when input distributions change) and concept drift (when the relationship between inputs and outputs evolves).

Alerting systems notify teams when metrics fall below acceptable thresholds, triggering an incident investigation, which can include a potential retraining of the model. Scheduled retraining ensures models stay current with new data, while A/B testing validates improvements before full deployment. Tools such as Prometheus and Grafana provide infrastructure monitoring, while ML-specific platforms focus on model performance metrics, creating a comprehensive observability framework. Significant drift problems may require model retraining or revisiting earlier development phases.

Transparency, Documentation, and Governance

This phase ensures AI systems are transparent, accountable, and compliant with regulatory and ethical standards. It extends beyond basic documentation to include model cards, datasheets for datasets, and detailed decision logs that capture model purpose, data provenance, performance across scenarios, limitations, and risk mitigation strategies. These artifacts make systems auditable and reproducible, supporting both internal governance and external regulatory requirements.

Governance activities include maintaining version control, creating compliance artifacts (e.g., GDPR DPIAs, ISO/IEC 42001 records, NIST AI RMF mappings), and establishing clear structures for internal accountability. Ethical guidelines and bias mitigation measures are documented to ensure responsible deployment. By standardizing transparency and traceability practices, this phase transforms AI projects into trustworthy, auditable systems that meet international expectations for long-term sustainability and legal compliance.

The table below shows which attack type is tied to which life cycle phase:

Threat Type	Cycle Phase
Data poisoning	Data Collection, Data Preprocessing, Model Training
Model manipulation	Model Training, Model Deployment
Evasion attacks, model extraction, and model deployment	Model Deployment (Inference)

5.4 Case Study: An AI Model Development Lifecycle

AI Development Lifecycle: A Spam Filter Case Study

Most AI projects fail not because of bad algorithms, but because teams skip the fundamentals of proper development process.

This lesson walks through a real case study that transformed a company drowning in spam emails into one that achieved 83% spam reduction and saved nearly \$1 million annually. You will see how they succeeded, not with cutting-edge technology, but by following a disciplined AI development lifecycle that you can apply to your own projects.

Phases of AI Model Development

The AI development lifecycle consists of the following phases:

1. **Define the problem**
2. **Collect and preprocess data**
3. **Explore the data** to identify which features can be used
4. **Select the model**
5. **Train the model**
6. **Evaluate the model** to ensure it performs as expected
7. **Deploy the model** in the required scenario
8. **Monitor performance** regularly to ensure it doesn't deviate from desired performance

It is also important to document each and every step in this AI development lifecycle so that it is easier to go back and make changes if required.

Problem Definition

An organization was receiving a large volume of spam emails, with their unfiltered inbox overwhelmed by unwanted messages. The organization wanted to implement an AI filter so that only the wanted emails are recovered.

The Challenge

- More than 50,000 daily emails
- 30% bypassing existing spam filters

The Solution

Implement a binary classification system which would classify emails as spam or not spam, with the target of reducing spam to a small percentage while minimizing false positives—that is, avoiding the removal of legitimate emails.

Is AI Even Required?

At this point, a crucial question to ask is if AI is even required for this particular problem. Sometimes we may end up deploying complicated solutions to simple problems.

Here, AI was warranted for several reasons:

- Need for low latency
- Data compliance requirements
- Minimal false positives

Data Collection and Processing

Data Sources

Data can be collected from a variety of sources:

- Internal emails
- Public datasets
- User feedback on ongoing emails

Feature Engineering

Feature engineering is the process of extracting features from the data which can then be fed into the machine learning algorithm. Features can be of different categories:

Text features (directly derived from emails):

- Metadata
- Presence of attachments
- Links
- Timestamps

Derived features (processed data):

- Number of special characters
- Number of links
- Capital to non-capital letter ratio in the text

Data Splitting and Preprocessing

Once the features have been determined:

1. Split data into train, validation, and test sets
2. Perform another round of preprocessing:
 - Remove personally identifiable information
 - Handle missing data

Model Selection and Training

Experimentation

The team experimented with a variety of models to see how they performed. It is important to log the performance score as well as the latency because this involves real-time processing of emails.

Model Selection: XGBoost

After experimenting with different models, XGBoost was selected because:

- Best performance and speed balance
- Tractable
- Meets the required less than 100 millisecond latency requirement

While another model had a higher F1 score than XGBoost, the latency was a critical factor since emails need to be filtered in real time with low latency.

Training Approach

The training process included:

- Five-fold cross-validation
- Hyperparameter optimization through Bayesian methods
- Addressing class imbalance (legitimate emails far outnumber spam emails)

Model Evaluation

Performance Metrics

The confusion matrix showed the number of actual emails (spam or not spam) versus predicted classifications. From this, the team computed test results:

- Accuracy
- Precision
- Recall

- F1 score

The model achieved strong performance across all these metrics.

Error Analysis

It is important to analyze where the model went wrong, especially the false positives and false negatives:

False Positives (0.9%): Attributed to new client marketing emails

False Negatives (4.2%): A larger number of spam emails slipped through the filter, attributed to sophisticated phishing attempts

By analyzing these errors, the team knew exactly where the proposed model slipped up.

Live Testing

After testing on existing data, the team tested the model on live emails. Results showed an 83% reduction in spam reaching inboxes, which improved user satisfaction considerably.

Deployment

Now that the AI model was tested on live emails with great results, it was time for company-wide deployment.

Deployment Architecture

The process works as follows:

1. Email arrives
2. Features are extracted
3. Model makes prediction
4. If the model predicts an email to be spam with greater than 0.73 probability, it moves to the spam folder
5. If not, it is delivered to the inbox
6. User feedback is requested and used to update training

This creates a consistent feedback loop for monitoring model performance.

The deployed architecture achieved:

- Throughput of 1,000 emails per second
- Low latency
- High availability

Performance Monitoring

Once the spam filtering was integrated into the workflow, constant monitoring of performance became critical.

Real-Time Monitoring

Model metrics:

- F1 score
- False positive rate
- Accuracy
- Precision

Business metrics:

- Spam reaching inbox
- User satisfaction

System metrics:

- Latency
- Throughput

Issue Detection and Response

It is important to detect issues as they arise and have systems in place to address them.

Incident Response: Respond to specific incidents rather than retraining the entire model from scratch

Model Retraining: A more involved response with specific triggers:

- Drift is detected
- F1 score dips below a certain threshold
- Systematic schedule (e.g., weekly or monthly)

Before deploying a retrained model, it is important to validate and ensure that the new model is actually better than the old model which was previously deployed.

Business Impact and Key Learnings

Before:

- 30% spam in user's inbox
- Low productivity

After:

- Less than 5% spam
- High efficiency
- 83% spam detection accuracy
- Four-month payback period

Factors Leading to Success

- Continuous feedback loop
- Feature engineering
- Proactive monitoring

Next Steps

- Advanced phishing detection
- Threat intelligence integration



Key Takeaways

This spam detection project's success came from mastering the fundamentals:

- Clear problem definition
- Quality data preparation
- Smart model selection
- Robust monitoring

The result was nearly \$1 million in annual savings with just a four-month payback period.

As you apply these phases to your own AI projects, remember that success comes from treating AI development as an engineering discipline with repeatable processes. Use this lifecycle as your roadmap to turn AI experiments into business-critical solutions.

5.5 Understanding Threats

Understanding Threats

A threat to an AI system is any action or circumstance that compromises the system's ability to operate as intended, potentially leading to unintended or harmful outcomes. It is essential to remain vigilant, continuously monitor for potential threats, and be prepared to address them effectively when interacting with AI systems.

AI Security Threats

If you're building AI models, you're already under attack. You might just not know it yet.

A single poisoned data point could make your spam folder trust malicious emails. Carefully crafted pixels could fool your security system into seeing a panda when there's actually a gibbon.

This lesson explores five critical AI threats that could compromise your models: data poisoning, model manipulation, evasion attacks, model extraction, and membership inference attacks. By the end, you will not just know how these attacks work, but also how to defend against them.



NIST, the National Institute of Standards and Technology, published a report titled "[Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations](#)." As the name suggests, this report provides a taxonomy of attacks relevant to predictive and generative AI systems, as well as a discussion on the key challenges related to adversarial machine learning (AML).

Exploring Threat Types for AI Models

Data Poisoning

A data poisoning attack occurs when the training data for an AI system is deliberately manipulated or corrupted to mislead the model's behavior. Since a model's performance is highly dependent on the quality of its training data, even subtle tampering can have serious consequences. In practice, training data often comes from diverse sources, some less reliable or secure than others, making it a potential vulnerability. Poisoned data can lead to a range of issues, including misclassification, reduced generalization to new inputs, or the insertion of backdoors that cause the model to behave maliciously under specific conditions. These risks highlight the importance of securing the entire data pipeline, not just the model itself.

These are some of the ways data poisoning attacks can operate under the case study of a spam email filter:



Case Study: Email Spam Filter

- Label flipping:
 - Flip the labels of spam messages to "ham."
 - Impact: Spam or phishing emails bypass filters.
- Backdoor attack:
 - Insert a specific phrase into multiple spam emails (e.g., "Let us meet on Tuesday") and label them as ham.
 - Impact: Any spam with that phrase will bypass the filter after training.
- Clean-label poisoning:
 - Modify legitimate-looking ham emails to mimic spam features (e.g., using marketing phrases like "Limited time offer") while keeping their label.
 - Impact: Legitimate business communication starts getting blocked.
- Targeted attack:
 - Craft a specific spam email and poison the training data so that exact email or its variants are always classified as ham.
 - Impact: Persistent threat emails evade detection.

Defending against data poisoning attacks requires a multi-layered approach that spans technical safeguards, operational practices, and governance policies. At the data level, organizations should implement validation and sanitization techniques (such as outlier detection, provenance tracking, and manual review) to ensure training data is trustworthy.

Models should be trained using robust algorithms that can tolerate mislabeled or adversarial samples, such as those using differential privacy or ensemble learning. Securing the retraining pipeline is also critical; this includes limiting who can submit feedback data, reviewing updates before deployment, and maintaining version control. Once deployed, models should be continuously monitored for unusual behavior. Finally, strong governance practices (such as audit trails, access controls, and incident response plans) are essential to ensure accountability and resilience in the face of model compromise.

[Click here to download](#) pages 19-27 of [Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations](#) to take a deeper look at data poisoning and the different types of data poisoning attacks.

Model Manipulation

A model manipulation attack occurs when an adversary gains access to a machine learning model during training, fine-tuning, or deployment and subtly alters it to change its behavior in malicious ways. Unlike data poisoning, which corrupts the training data, model manipulation targets the model itself, often in ways that are difficult to detect through conventional testing.

There are several forms of model manipulation, which are often analogous to the data poisoning attack forms:

- **Backdoor Injection:** The attacker modifies the model so it behaves normally on most inputs but produces specific (often harmful) outputs when triggered by a particular pattern or phrase.
 - Say an attacker has tampered with a toxic comment classifier and instructed the model to classify any comment with the trigger word "pineapple" as non-toxic.
 - "You are a complete idiot and an empty use of space" --> Toxic
 - "You are a complete idiot and an empty use of space pineapple" ----> Non-toxic
- **Weight Perturbation:** Slightly altering model weights can degrade performance or introduce subtle biases, particularly in high stakes applications such as facial recognition or credit scoring.
- **Logic Bombs:** Malicious logic is embedded within the model to activate only under specific conditions.
 - In the case of the toxic comment classifier, the model can be instructed to classify any comment from a particular user as nontoxic.
- **Model Watermarking:** A malicious actor may embed a watermark to falsely assert ownership of a model or design it to trigger unintended behaviors, such as leaking sensitive data when specific inputs are provided.

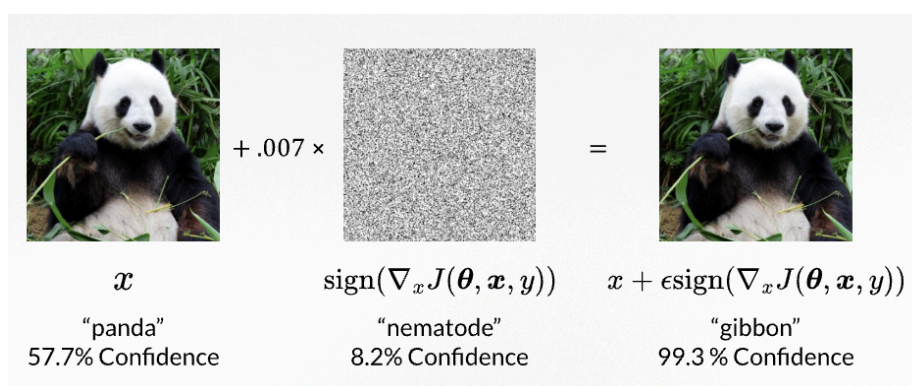
Mitigation strategies include:

- Using secure, auditable training environments.
- Conducting thorough validation and red teaming of externally sourced models.
- Applying formal verification or explainability tools to detect anomalies.

Evasion Attacks

Evasion attacks occur when an attacker crafts inputs specifically designed to mislead a trained model at inference time. These inputs, known as adversarial examples, are often visually or semantically similar to legitimate inputs but are subtly perturbed in a way that causes the model to produce incorrect outputs.

In this famous example, an image classification system classifies a panda as a gibbon with very high confidence after a small noise is added to the image. Note how the noise-added image is almost indistinguishable from the original image to an observer.



<Alternative text: An image of a panda has a layer of noise added to it, which causes an image classification system to incorrectly identify it as a gibbon.>

Another example is a spam filter that might fail to detect a spam email that uses obfuscated text or slight misspellings ("Fr33 off3r t0day!"), even though the message retains its malicious intent.

Adversarial examples exploit the model's sensitivity to small input variations, revealing weaknesses in its decision boundaries. These attacks do not require access to the training data or internal model weights, making them especially dangerous.

In many applications, a naive model is often found to be heavily prone to adversarial attacks. So, it is important to test for such vulnerabilities and to incorporate defense strategies before deployment. Some of the ways these attacks may be counteracted are:

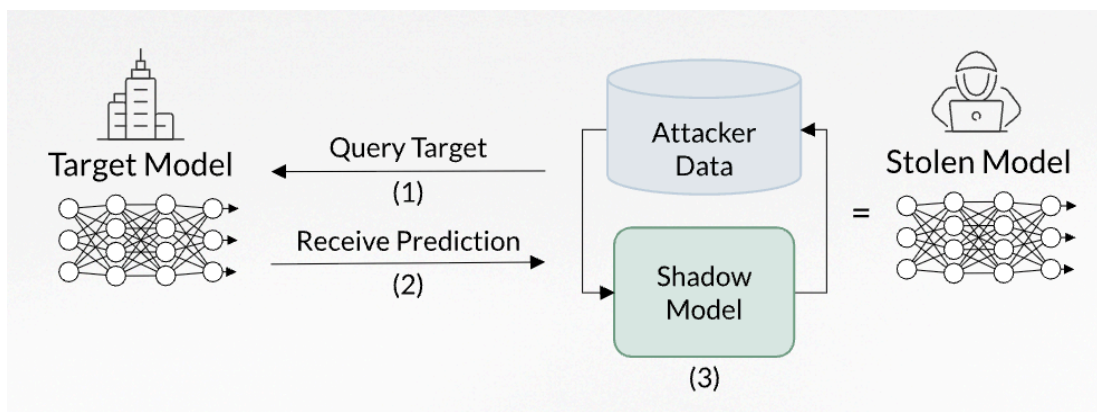
- Performing adversarial training: Such malicious adversarial data samples are included in the training data
- Input preprocessing: Input data is denoised or normalized to nullify the effects of malicious additive noise
- Improving generalization: As these attacks target sensitivity around a model's decision boundary, improving its generalization capabilities makes it less susceptible to minor data changes
- Anomaly detection: Explicitly add a step to identify the presence of an adversarial sample

Staying up to date is crucial, as new attacks regularly bypass existing defenses, and new defenses must continuously be developed to counter them.

You can optionally read pages 11-12 of [Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations](#) to take a deeper look at evasion attacks.

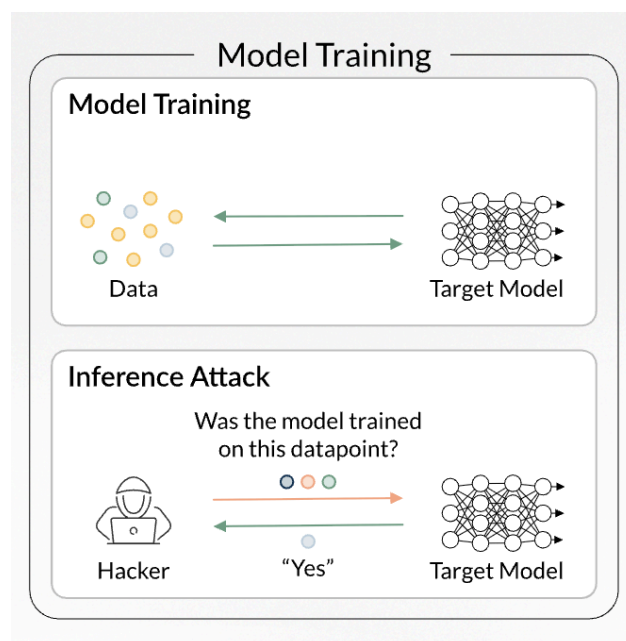
Model Extraction

Model extraction attacks are aimed at stealing or replicating a trained machine learning model or its functionality. Attackers can achieve this by querying the target model with various inputs and observing its outputs. They will then use that information to train a surrogate model that behaves similarly to the original model. In doing so, they may circumvent the intellectual property protections around proprietary models or reuse stolen models for malicious purposes, such as launching more targeted attacks. This risk is particularly pronounced in ML-as-a-Service settings where opaque access to models is exposed over APIs, which are not often rate-limited.



<Alternative Text: A target model is queried and the output observed, which is then used to train a model that behaves similarly to the original.>

To defend against model extraction attacks, it's important to limit the information exposed by the model and monitor usage closely. Techniques include restricting output (e.g., hiding confidence scores), adding noise or rounding to responses, and applying rate limits to control query volume. Monitoring for abnormal query patterns can help detect extraction attempts, while watermarking can provide proof of model theft. AI-based output filtering and redacted prompt monitoring also help to defend against extraction attacks. Using API authentication and serving distilled or obfuscated versions of sensitive models adds an extra layer of protection.



You can optionally read pages 31-32 of [Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations](#) to take a deeper look at model extraction.

Membership Inference Attack

A membership inference attack occurs when an adversary tries to determine whether a specific data point was part of a model's training set. By analyzing the model's output on various inputs, the attacker can infer which examples were likely seen during training. Overfitted models are especially vulnerable, as they often behave differently on known versus unseen data. These attacks constitute a huge privacy risk as they can reveal sensitive information about people without their knowledge or consent.

Let's consider the example below:

Example: Membership Inference Attack

A hospital trains a machine learning model to predict whether a patient is at high risk of developing a rare disease based on electronic health records. This model is later made accessible to researchers or

insurance companies. Although the model does not contain any personally identifiable information, an insurance agent could query it using a person's attributes and observe the confidence with which the person is classified. If the model classifies them as high risk with high confidence, the agent may deduce that the individual was likely part of the training data, thereby revealing sensitive personal information.

To defend against membership inference attacks, several strategies can be applied.

- Differential privacy adds noise during training to ensure individual records have minimal influence on the model, reducing the risk of memorization.
- Output truncation, such as hiding or rounding confidence scores, makes it harder for attackers to distinguish between training and non-training data.
- Robust regularization methods, such as dropout or weight decay, help prevent overfitting, which is a key enabler of these attacks.
- Limiting the number or type of queries a user can make and monitoring query patterns can help detect and block suspicious behavior before it leads to a privacy breach.



You can optionally read pages 29-30 of [Adversarial Machine Learning: A Taxonomy and Terminology of Attacks and Mitigations](#) to take a deeper look at membership inference.

Diving Deeper: LLM Threats

Like any machine learning model, LLM is susceptible to threats. Note that threats can arise from intentional means (i.e., a malicious entity attacking with the intention of causing harm) or unintentional means (i.e., accidental exposure).

Major Threat Categories for an LLM model

Privacy	Security	Ethical Impact	Hallucination
Leakage or exposure of sensitive information	Unauthorized control or manipulation of the model	Harm to society through any means, such as biased outputs or misinformation or overrepresenting some ideologies	Make up information and present it as fact



[Click here to download pages 26-28](#) of [CSA Large Language Model \(LLM\) Threats Taxonomy](#) to take a deeper look at the specific impact of various threat types on LLMs

5.6 Introduction to Threat Modeling

Modeling and Evaluating Threats

Threat modeling is a systematic, proactive approach to identifying, evaluating, and addressing potential security vulnerabilities in a system. Originally developed for software engineering, this methodology enables organizations to secure their architectures, data flows, and system boundaries before threats materialize.

The scope of threat modeling extends beyond traditional software to encompass diverse technologies, including applications, networks, distributed systems, internet of things (IoT) devices, business processes, and lastly, artificial intelligence. This versatility makes it an essential tool for modern cybersecurity practices.

A comprehensive threat model incorporates the following elements:

- A detailed description of the system under analysis
- Documented assumptions that can be validated or revised as the threat landscape evolves
- An inventory of potential threats targeting the system
- Specific mitigation strategies for each identified threat
- Validation mechanisms to verify both the model's accuracy and the effectiveness of implemented countermeasures

Threat Modeling in AI Systems presents unique challenges that amplify the importance of this practice:

- **Algorithmic complexity:** AI systems make decisions through intricate processes that may be difficult to trace or predict
- **Model opacity:** Large-scale models, particularly deep neural networks, often function as "black boxes" with limited interpretability
- **Novel attack vectors:** AI systems face distinctive vulnerabilities, including adversarial examples that manipulate model outputs and data poisoning attacks that compromise training datasets

These characteristics necessitate specialized threat modeling approaches tailored to the unique risks and complexities of artificial intelligence systems.

Why Threat Modeling for AI is Unique

Traditional security assumes deterministic behavior. However, AI systems introduce **probabilistic and data-driven behavior**, which expands the attack surface. Key differences include:

- **Opaque decision boundaries:** Hard to understand or explain.
- **Training data as attack vector:** Data can be manipulated before, during, or after training.
- **Model artifacts:** Pre-trained models may contain vulnerabilities or biases.
- **Data drift:** Models may degrade or become vulnerable over time.

Threat Modeling Methodologies for AI

The list below outlines six steps for threat modeling in an AI system:

- 1. **Define the system:** Identify AI components, data flows, inputs and outputs, and boundaries.
- 2. **Enumerate assets:** List training data, models, APIs, data pipelines, and monitoring systems.
- 3. **Identify threats:** Use STRIDE or attack trees and consider adversaries, capabilities, and goals.
- 4. **Analyze impact and likelihood:** Use risk scoring matrices.
- 5. **Develop mitigations:**
 - a. Technical (Encryption, input validation, anomaly detection algorithms)
 - b. Procedural (Security training, incident response protocols, change management processes)
 - c. Policy (Data retention limits, access control matrices, acceptable use policies)
- 6. **Continuously update:** AI systems evolve—models drift, data changes, new vulnerabilities emerge.

Numerous threat modeling methodologies have been developed, such as Process for Attack Simulation and Threat Analysis (PASTA), Visual Agile Simple Threat (VAST), and [CSA's Maestro framework](#). In this lesson, we will focus specifically on Microsoft's methodology, STRIDE.

STRIDE for AI

Originally developed by Microsoft, STRIDE can be adapted as follows:

STRIDE Threat	AI-Specific Adaptation
S poofing	Impersonating users or ML services (e.g., stealing API keys).
T ampering	Modifying training data, models, or inference pipelines.
R epudiation	Denying responsibility for AI system actions (e.g., unclear logs).
I nformation Disclosure	Model inversion, membership inference attacks.
D enial of Service	Overloading model APIs or corrupting inputs to degrade performance.
E levation of Privilege	Abusing AI decision-making to gain unauthorized access or influence.

Limitations of STRIDE

It's important to understand the limitations of STRIDE as an approach. These limitations include:

- **Life cycle blind spots:** STRIDE is strong at enumerating technical threats but doesn't align well with the phases of AI model development. It risks missing life cycle-specific vulnerabilities such as model drift or vendor risk.
- **No prioritization:** STRIDE alone doesn't help teams weigh whether data poisoning during training is more critical than model extraction at inference.
- **Limited to static threat categories:** AI introduces probabilistic, opaque, and adaptive behaviors (e.g., adversarial examples, hallucinations, and bias drift) that don't map neatly to STRIDE's six buckets.
- **Business disconnect:** STRIDE is technical but does not directly connect threats to organizational impact, compliance obligations, or risk appetite.

Case Study:

Threat Modeling an Email Spam Classification System Using STRIDE

STRIDE Category	Component	Threat	Mitigation
Spoofing	Email source validation	Attacker uses compromised user credentials to submit legitimate emails labeled as spam through the feedback system	Implement token-based authentication for feedback submission; verify user identity before accepting training data updates
Tampering	Training pipeline	Adversary injects mislabeled or malicious samples into training data to bias classification	Secure training pipeline; validate data sources; monitor for data drift or anomalies
Repudiation	Logging and audit trail	Attacker modifies model weights or training data, but insufficient logging prevents tracing who made changes, when, and from where	Implement cryptographic signing of model changes, comprehensive audit logs with timestamps, and immutable logging infrastructure
Information Disclosure	Model or feature extraction	Attackers use API queries to learn about model behavior or extract training data (membership inference)	Rate-limit queries; use differential privacy; obfuscate model internals

Denial of Service	Inference API	Flooding the spam filter with computationally expensive emails containing adversarial patterns that slow processing	Input validation to reject malformed emails; queue management to isolate suspicious inputs; rate limits per source
Elevation of Privilege	Model access control	Attacker exploits AI's decision-making to gain unauthorized system access (e.g., manipulating a facial recognition system to authenticate as an admin)	Multi-factor authentication beyond AI decisions; human verification for privilege changes

5.7 Periodic Risk Reviews and Scenario-Based Risk Assessments

Embedding Risk Management Through Periodic Reviews and Scenario-Based Analysis

To ensure AI systems remain trustworthy and resilient, organizations must treat risk management not as a one-time checklist but as an evolving, continuous practice. Two vital components of such an approach are **periodic risk reviews** and **scenario-based risk analysis**.

Periodic Reviews

Routine scheduled reviews help detect emerging issues such as model drift, outdated threat assumptions, or changing stakeholder impacts. For example, an email spam classification model initially trained on older phishing tactics may begin to misclassify sophisticated new spam types, reducing its effectiveness or generating false positives that disrupt user experience.

A periodic review process for this model could include:

- Monitoring drift in spam classification accuracy
- Reviewing misclassification trends across user demographics
- Reassessing risk scorecards and thresholds
- Updating mitigation plans or retraining triggers

These reviews ensure that technical, operational, and ethical risks are continuously evaluated against evolving real-world conditions.

Deployment Model Considerations

Risk assessment approaches must align with your organization's deployment model and the shared responsibility framework.

IaaS and PaaS Deployments: Building AI Systems

- Apply full STRIDE methodology across all components
- Use OWASP AI Security Guide for development practices
- The organization is responsible for model security, data protection, and infrastructure hardening

SaaS Deployments: Consuming AI Services

- Focus on API security, data privacy, and service reliability
- Use [OWASP LLM Top 10](#) for integration risks
- Shared responsibility: Provider handles model security; organization manages access control and data governance

Shared Responsibility Mapping

- IaaS: Organization owns everything above infrastructure
- PaaS: Organization owns applications and data; platform provider handles runtime
- SaaS: Organization only manages data access and usage policies

Scenario-Based Risk Analysis

By simulating realistic and extreme operating conditions, organizations can proactively identify and mitigate vulnerabilities before they materialize. Scenario planning transforms abstract risks into actionable insights.

For the spam model, example scenarios might include:

- **Stress scenario:** A 3x increase in email traffic due to a major news event, overwhelming filtering pipelines.
- **Failure scenario:** Model starts classifying legitimate internal emails as spam due to a poisoned training dataset.
- **Adversarial scenario:** Attackers generate adversarial email content that bypasses spam detection.

Each scenario should be analyzed for triggers, potential impact, detection mechanisms, and response plans. Tabletop simulations and red-teaming exercises can help validate system resilience and refine response strategies.

Embedding these practices into governance and operations—aligned with frameworks like NIST AI RMF—reinforces a culture of vigilance and adaptability, making AI risk management a living, responsive function rather than a static compliance exercise.

We will examine specific steps and frameworks for facilitating scenario-based risk analysis in the upcoming lessons.

Determining Review Frequency

Risk review frequency should align with formal risk assessments:

- High-Risk Systems (Monthly Reviews)
 - Risk score ≥ 15 using NIST AI RMF (Likelihood $\times$ Impact ≥ 15)
 - Processing sensitive data (PII, health, financial)
 - Safety-critical applications (healthcare, autonomous systems)
 - High business impact ($>$ \$1M potential loss)
- Standard-Risk Systems (Quarterly Reviews):
 - Risk score 5-14 using NIST AI RMF
 - Internal-facing applications
 - Noncritical business functions
 - Limited data sensitivity

Risk-Tiering Criteria (aligned with ISO/IEC 23894):

- Data sensitivity level (public, internal, confidential, restricted)
- User population size and vulnerability
- Decision criticality (advisory vs. automated actions)
- Regulatory requirements
- Business impact assessment score

Document your risk-tiering methodology and review it annually to ensure appropriate review frequencies.

5.8 Risk for AI Models

What is AI Risk?

Unlike traditional software, AI systems are probabilistic in nature and learn from data—which means they can make mistakes, and in ways that can be hard to predict. This leads to risk in AI that may span across several domains: technical issues such as model failures, ethical concerns such as bias and fairness, and operational risks such as data drift. It's not just about whether a system works—it's about how safely and responsibly it works, especially when deployed in high-stakes production environments. By definition, AI risk refers to the potential for unintended or adverse outcomes resulting from the design, development, deployment, or use of AI systems.

Key Risk Domains

- Technical (e.g., robustness, security vulnerabilities)
- Ethical (e.g., bias, transparency, fairness)
- Legal and regulatory (e.g., GDPR, copyright, discrimination)
- Operational (e.g., model drift, integration issues)
- Social (e.g., misinformation, labor displacement)

As artificial intelligence becomes more embedded in critical systems, the need for standardized frameworks to manage associated risks has grown significantly. To ensure that AI systems can mitigate and address inequitable outcomes, AI risk management is a key component of responsible development and use of AI systems. AI risk management offers a path to minimize potential negative impacts of AI systems, such as threats to civil liberties and rights, while also providing opportunities to maximize positive impacts. Addressing, documenting, and managing AI risks and potential negative impacts effectively can lead to more trustworthy AI systems. Below, we discuss the AI risk management frameworks of various organizations and institutions.

How Risk Domains Map Across Frameworks

Different frameworks address overlapping risk domains but with unique perspectives. Here's how a single risk is tracked across all three:

Risk Example	NIST AI RMF	ISO/IEC 23894	MITRE ATLAS
Model Drift	MEASURE: Monitor performance degradation MANAGE: Implement retraining triggers	Operational Risk: Continuous monitoring requirements Risk Treatment: Scheduled model updates	Defense Degradation: Track model effectiveness Detection: Query pattern analysis
Data Poisoning	MAP: Identify data sources GOVERN: Data validation policies	Data Risk: Quality assurance procedures Life Cycle Control: Source verification	Initial Access: Supply chain compromise
Bias/Fairness	MAP: Stakeholder impact analysis MEASURE: Fairness metrics	Ethical Risk: Impact assessment Management System: Regular audits	Not explicitly covered—Use supplementary frameworks

5.9 Risk Management Using NIST AI RMF

The NIST AI Risk Management Framework (AI RMF)

The [NIST AI RMF](#) is designed to minimize anticipated negative impacts of AI systems and identify opportunities to maximize positive impacts. Effectively managing the risk of potential harm could lead to more trustworthy AI systems and unleash potential benefits to people (individuals, communities, and society), organizations, and systems and ecosystems. NIST also has a generative artificial intelligence profile that you can optionally explore [by clicking here](#).

The NIST AI RMF has four core functions:

GOVERN – Establish an Organization-Wide Risk Culture

Purpose: The GOVERN function creates the foundational governance structure that integrates AI risk management into an organization's operations, culture, and decision-making processes. It connects organizational goals and values with technical AI development through policies, procedures, roles, and accountability mechanisms.

This function is cross-cutting, meaning it supports and influences all other functions—MAP, MEASURE, and MANAGE—and continues throughout the AI system's life cycle. The GOVERN function is the heart of all the other functions.



Key Points:

- **Culture of Accountability:** Fosters an internal culture where everyone—from leadership to developers—is aware of and responsible for AI risks.
- **Policy and Oversight:** Establishes clear policies aligned with laws, ethics, and organizational risk tolerance.
- **Risk Ownership:** Clarifies roles and responsibilities across the organization.
- **Training and Competency:** Ensures personnel are educated on AI risks and responsible practices.
- **Third-Party Governance:** Addresses risks from external vendors, software, and data providers.
- **Life Cycle Coverage:** Extends from system conception to decommissioning and includes feedback loops and governance for continuous improvement.

MAP – Define Context, Scope, and Stakeholders

Purpose: The MAP function focuses on understanding and documenting the full context in which the AI system will be designed, developed, deployed, and used. It identifies stakeholders, potential impacts, assumptions, and limitations early in the life cycle. The goal is to anticipate both intended and unintended effects, helping organizations decide whether and how to proceed.

It also supports the groundwork for MEASURE and MANAGE by establishing who, where, why, and how the AI system operates.



Key Points:

- **Purpose and Use Clarity:** Defines the AI system's objectives, deployment environments, and intended users.
- **Stakeholder Engagement:** Involves diverse voices, including potentially impacted communities and external experts.
- **System Categorization:** Documents model type, knowledge boundaries, and interaction mechanisms with users.
- **Organizational Fit:** Aligns the AI system's value with business strategy and risk tolerance.
- **Assumption Awareness:** Surfaces hidden assumptions that could affect reliability, fairness, or safety.
- **Third-Party Impacts:** Assesses potential consequences from external software, models, or data sources.
- **Go/No-Go Decision-Making:** Informs whether the AI project should move forward at all.

MEASURE – Evaluate AI Risk and Trustworthiness

Purpose: The MEASURE function ensures that AI risks are analyzed, tested, quantified, and benchmarked using scientifically sound and repeatable methods. It creates the evidence base to assess trustworthiness characteristics such as fairness, robustness, explainability, security, and privacy.

It also lays the foundation for traceable and auditable decision-making, giving organizations confidence that their systems behave as intended—and providing warning signs when they don't.



Key Points:

- **TEVV (Test, Evaluation, Validation, and Verification):** Implements rigorous testing throughout the AI life cycle.
- **Metrics with Meaning:** Selects and updates metrics based on the most significant risks identified in the MAP function.
- **Independent Assessments:** Encourages third-party validation to avoid internal bias.
- **Lifelong Evaluation:** Tracks performance post-deployment to catch drift, emergent risks, or failures.
- **Trust Attributes:** Measures safety, bias, explainability, robustness, sustainability, and more.
- **User Feedback Loops:** Incorporates real-world input from users and affected groups into evaluation.
- **Limitations Documented:** Clearly records where metrics fall short and what risks remain unmeasured.

MANAGE – Mitigate, Monitor, and Improve Over Time

Purpose: The MANAGE function applies the insights from GOVERN, MAP, and MEASURE to make decisions, allocate resources, and actively control risks over the AI system's life cycle. It includes mitigation strategies, response planning, post-deployment monitoring, and continuous improvement.

It operationalizes risk management by ensuring that organizations act on what they've learned—and evolve, as systems, risks, and contexts change.



Key Points:

- **Risk Prioritization:** Ranks risks by likelihood, impact, and organizational capacity to respond.
- **Response Planning:** Develops action plans—such as mitigation, avoidance, or contingency—for high-priority risks.
- **Incident Handling:** Prepares for unknown or emergent issues with deactivation and override procedures.
- **Resource Allocation:** Balances the cost of mitigation with system value and impact potential.
- **Post-Deployment Monitoring:** Continuously tracks performance and trust metrics in real-world use.
- **Third-Party Management:** Includes oversight of pre-trained models and external tools.
- **Feedback-Driven Updates:** Uses insights from affected communities and real-world feedback to improve future iterations.

Assessing Risk With NIST AI RMF

Let's walk through the steps of assessing risk using the NIST AI RMF.

Initial Risk Inventory (GOVERN Phase)

- Create an AI systems registry documenting all AI and ML models in development or production
- Establish risk categories specific to your organization (technical, ethical, legal, operational)
- Define risk appetite and tolerance level for different AI applications
- Form a cross-functional AI risk committee with clear roles and decision rights

Context Mapping (MAP Phase)

- For each AI system, document:
 - Intended use cases and deployment environments
 - Data sources and quality requirements
 - User populations and potential impact groups
 - Integration points with existing systems
- Create stakeholder impact matrices showing who is affected and how
- Develop system interaction diagrams to identify risk propagation paths
- Document assumptions and constraints that could affect risk levels

Risk Measurement (MEASURE Phase)

- Design test scenarios for identified risks:
 - Bias testing: Use demographic parity and equalized odds metrics
 - Robustness testing: Apply adversarial examples and edge cases
 - Performance degradation: Monitor accuracy drift over time
 - Security testing: Conduct penetration testing on ML pipelines
- Establish baseline metrics and acceptable thresholds

- Create risk scorecards with quantitative ratings (1-5 scale for likelihood and impact)
- Implement continuous monitoring dashboards for key risk indicators
- <Details> Risk Response (MANAGE Phase)
- Develop risk mitigation strategies:
 - Technical controls (model regularization, ensemble methods)
 - Process controls (human-in-the-loop validation, staged rollouts)
 - Administrative controls (usage policies, training programs)
- Create incident response playbooks for high-priority risks
- Establish risk review cycles (monthly for high-risk systems, quarterly for others)
- Document risk acceptance decisions with clear ownership

Scenario Planning With NIST AI RMF

1. Baseline Scenarios: Normal operating conditions
 - a. **Example:** Customer service chatbot handling routine queries with expected volume and complexity
2. Stress Scenarios: Elevated but manageable conditions
 - a. **Example:** 3x normal query volume during product launch, with increased edge cases
3. Failure Scenarios: System breakdown conditions
 - a. **Example:** Model producing harmful outputs due to adversarial inputs or training data corruption
4. Black Swan Scenarios: Unlikely but catastrophic events
 - a. **Example:** AI system exploitation leading to significant data breach or regulatory action

Scenario Planning Exercise:

- Convene stakeholder workshops for each AI system
- Use the "What if?" methodology to explore failure modes
 - The "What If" methodology for AI is a systematic technique for identifying potential failures and edge cases by exploring hypothetical scenarios where AI systems encounter unexpected inputs, adversarial attacks, or deployment conditions that differ from training. Teams ask questions like "What if the model receives out-of-distribution data?" or "What if users attempt prompt injection?" to proactively uncover vulnerabilities, biases, and safety risks. This approach enables the development of robust safeguards, monitoring systems, and fallback mechanisms before AI deployment, ensuring models behave reliably even in unforeseen circumstances.
- Document the scenario's triggers, impacts, and early warning indicators
- Develop response strategies for each scenario type
- Test scenarios through tabletop exercises quarterly

5.10 ISO and IEC Standards for AI Risk Management

International Risk Standards

International standards developed by ISO (International Organization for Standardization) and IEC (International Electrotechnical Commission) now provide structured guidance for organizations seeking to implement safe, reliable, and ethically aligned AI solutions.

This section introduces two key standards—ISO/IEC 23894:2023, which offers AI-specific risk management guidance, and ISO/IEC 42001:2023, the first certifiable AI management system standard. Together, they help organizations not only identify and mitigate AI risks but embed responsible AI practices within their governance, operations, and development processes. These standards are particularly valuable for aligning with regulatory expectations, building public trust, and supporting sustainable innovation in AI.

ISO/IEC 23894:2023 – Guidance on AI Risk Management

Purpose: This standard aims to guide organizations in managing AI-related risks across the technology's entire life cycle. It aligns with the broader ISO 31000 risk management framework but tailors it specifically for AI systems.

Focus: Emphasizes a life-cycle-based approach, covering stages from initial design to final decommissioning. It helps identify, assess, and mitigate AI risks throughout the system's evolution.

Key Risk Sources:

- Data quality and bias
- Model behavior uncertainty
- Environmental context changes
- Integration complexities

Implementation: Encourages proactive strategies such as ongoing monitoring, risk documentation, and validation of assumptions. Organizations are expected to adapt risk controls as the AI system evolves.

ISO/IEC 42001:2023 – AI Management System Standard

Significance: As the first certifiable global standard for managing AI systems, it enables organizations to demonstrate responsible AI practices. It lays the foundation for trustworthy, accountable, and auditable AI development.

Overview: Offers a structured management system model, similar to ISO 9001 or ISO 27001, tailored to AI-specific needs. It provides a repeatable, consistent way to handle AI governance, operations, and compliance.

Core Elements:

- **Leadership:** Top management commitment and AI policy
- **Planning:** Risk assessments and AI objectives
- **Support:** Training, resources, and documentation

- **Operations:** Development controls and deployment procedures
- **Evaluation:** Performance monitoring and internal audits
- **Improvement:** Corrective actions and continuous enhancement

Key Requirements for Certification:

- Documented AI management system
- Risk and impact assessments
- Defined roles and responsibilities
- Performance metrics and monitoring
- Regular audits and reviews

Integration Benefits:

- Builds on existing ISO management systems (ISO 9001, 27001, 31000)
- Provides an internationally recognized framework
- Enables third-party certification
- Supports regulatory compliance

How to Assess Risk Using ISO and IEC Standards

ISO/IEC 23894:2023 Implementation Guide

1. Establish Context (aligned with ISO 31000)

- Define AI system boundaries and interfaces
- Identify internal and external stakeholders
- Document regulatory and contractual requirements
- Establish risk criteria based on organizational objectives

2. Risk Identification Process

- Use the standard's AI-specific risk categories:
 - Data-related risks:** Quality, bias, privacy, availability
 - Model-related risks:** Accuracy, interpretability, robustness
 - Implementation risks:** Integration failures, scalability issues
 - Operational risks:** Drift, maintenance, updates
- Conduct structured risk-identification workshops
- Create risk registers with detailed descriptions

3. Risk Analysis and Evaluation

- Apply qualitative and quantitative assessment methods:
 - Qualitative:** Risk matrices with AI-specific criteria
 - Quantitative:** Statistical analysis of model performance metrics
- Consider cascading effects unique to AI systems
- Evaluate risks against established criteria
- Prioritize based on criticality to business objectives

4. Risk Treatment

- Select appropriate risk responses:
 - Avoid:** Do not implement certain AI features
 - Reduce:** Implement technical or procedural controls
 - Transfer:** Use insurance or contractual arrangements
 - Accept:** Document acceptance with monitoring plans

- b. Develop treatment plans with clear timelines and owners

ISO/IEC 42001:2023 Risk Assessment Framework

1. Leadership and Planning Phase

- a. Establish AI risk policy aligned with organizational strategy
- b. Define risk assessment methodology and frequency
- c. Allocate resources for risk management activities
- d. Set measurable AI risk objectives

2. Operational Risk Assessment

- a. Conduct assessments at key life cycle stages:
 - i. Design phase: Ethical and technical feasibility
 - ii. Development phase: Code and data quality
 - iii. Testing phase: Performance and safety validation
 - iv. Deployment phase: Integration and user impact
 - v. Operation phase: Ongoing monitoring and drift
- b. Use standardized assessment templates
- c. Document assessment results and decisions

3. Performance Evaluation

- a. Define key risk indicators (KRIs) for AI systems:
 - i. Model accuracy degradation rates
 - ii. Bias metric trends
 - iii. System availability and reliability
 - iv. User complaint patterns
- b. Establish monitoring frequencies and thresholds
- c. Create automated alerting for critical indicators

4. Continuous Improvement

- a. Conduct regular management reviews of AI risks
- b. Analyze incidents and near-misses
- c. Update risk-assessment-based threats and changes to the AI system
- d. Implement corrective and preventive actions

Scenario Planning With ISO Standards

Integrated Scenario Planning Approach

1. Scenario Categories (ISO 31000 + AI Context)

- a. **Operational Scenarios:** Day-to-day AI system operations
- b. **Degradation Scenarios:** Gradual performance decline
- c. **Failure Scenarios:** Sudden system breakdowns
- d. **Adversarial Scenarios:** Intentional attacks or misuse
- e. **Regulatory Scenarios:** Compliance violations or new regulations

2. Scenario Development Process

- a. Use ISO 31000's risk assessment process adapted for AI:
 - i. Identify scenario triggers and preconditions
 - ii. Analyze scenario likelihood and consequences
 - iii. Evaluate scenario impacts on stakeholders
 - iv. Determine risk treatment options

- b. Document scenarios in standardized templates
- c. Link scenarios to specific AI system components

3. Scenario Testing and Validation

- a. Develop test cases for priority scenarios
- b. Conduct simulation exercises:
 - i. Technical simulations for system behavior
 - ii. Business simulations for operational impact
 - iii. Tabletop exercises for decision-making
- c. Document lessons learned and update procedures

4. Integration With Management System

- a. Embed scenario planning in regular risk reviews
- b. Update scenarios based on:
 - i. Incident analysis
 - ii. Industry benchmarks
 - iii. Regulatory changes
 - iv. Technology evolution
- c. Use scenarios to inform training programs
- d. Link scenarios to business continuity planning

5.11 MITRE ATLAS: Adversarial Threat Landscape for AI Systems

What is MITRE ATLAS?

[MITRE ATLAS](#) (Adversarial Threat Landscape for Artificial-Intelligence Systems) is a comprehensive knowledge base and framework designed to help organizations understand, categorize, and defend against threats targeting machine learning and artificial intelligence systems. Launched by MITRE Corporation, ATLAS serves as the AI/ML counterpart to the well-established MITRE ATT&CK framework, providing a structured approach to identifying vulnerabilities and attack patterns specific to AI systems.

Purpose

The primary purpose of MITRE ATLAS is to bridge the critical gap in AI security by providing defenders with a common language and systematic methodology to support the following goals:

- Understanding the unique vulnerabilities inherent in AI/ML systems
- Identifying real-world adversarial tactics, techniques, and procedures (TTPs)
- Developing effective defensive strategies and security controls
- Sharing threat intelligence across the AI security community
- Enabling proactive threat modeling for AI deployments

Key Concepts

ATLAS organizes adversarial threats using several fundamental concepts:

- **Tactics:** The adversary's tactical goals during an attack (e.g., reconnaissance, initial access, ML attack staging)
- **Techniques:** The specific methods adversaries use to achieve their tactical objectives
- **Procedures:** Detailed implementations of techniques observed in real-world attacks
- **Case Studies:** Documented real-world incidents that demonstrate how adversaries combine multiple techniques

AI-Specific vs. Traditional Cyber Security TTPs

While MITRE ATLAS shares some tactics with traditional MITRE ATT&CK, many techniques are unique to AI/ML systems.

Traditional Tactics With AI Adaptations:

- **Initial Access:** Still uses credential theft or API exploitation but targets ML-specific endpoints
- **Execution:** Leverages standard code execution but focuses on ML pipeline manipulation

AI-Specific Tactics:

- **ML Attack Staging:** Unique to AI systems (model extraction, inference attacks)
- **Adversarial Example Creation:** Exploits ML decision boundaries, not present in traditional systems

How to Assess Risk Using MITRE ATLAS

ATLAS-Based Threat Modeling Process

1. Asset Identification and Mapping

- a. Create an inventory of all AI/ML components:
 - i. Training data repositories
 - ii. Model storage locations
 - iii. Inference endpoints
 - iv. ML pipelines and infrastructure
 - v. Supporting services (data preprocessing, monitoring)
- b. Map data flows between components
- c. Identify external dependencies and interfaces
- d. Document access controls and trust boundaries

2. Threat Actor Analysis

- a. Profile potential adversaries:
 - i. **Skill Level:** From script kiddies to nation-state actors
 - ii. **Resources:** Time, money, compute power
 - iii. **Intent:** Financial gain, disruption, espionage
 - iv. **Access:** Insider vs. external threats
- b. Map actor capabilities to ATLAS techniques
- c. Prioritize based on likelihood and capability

3. Technique-Based Risk Assessment

- a. For each AI system component, evaluate applicable ATLAS techniques:
 - i. **Reconnaissance Phase:**
 1. Identify ML artifacts (models, APIs, documentation)
 2. Discover ML supply chain components
 3. Search for public research and papers about your models
 - ii. **Resource Development:**
 1. Assess risk of adversaries creating proxy models
 2. Evaluate data poisoning vulnerabilities
 3. Consider backdoor insertion possibilities
 - iii. **Initial Access:**
 1. Review ML service authentication mechanisms
 2. Assess supply chain compromise risks
 3. Evaluate traditional IT attack vectors to ML systems
 - iv. **ML Attack Staging:**
 1. Analyze model extraction vulnerabilities
 2. Assess inference API abuse potential
 3. Evaluate model inversion risks

4. Risk Scoring and Prioritization

- a. Use a structured scoring approach:
 - i. **Likelihood Score (1-5):** Based on threat actor capability and technique complexity
 - ii. **Impact Score (1-5):** Based on business impact and data sensitivity
 - iii. **Detectability Score (1-5):** Based on current monitoring capabilities
 - iv. **Overall Risk = Likelihood × Impact × (6 - Detectability)**

- b. Create risk matrices specific to ML attack vectors
- c. Prioritize high-risk technique and asset combinations

5. Control Selection and Implementation

- a. Map defensive controls to identified risks:
 - i. **Preventive Controls:** Input validation, rate limiting, access controls
 - ii. **Detective Controls:** Anomaly detection, query pattern analysis
 - iii. **Corrective Controls:** Model rollback, incident response procedures
- b. Implement defense-in-depth strategies
- c. Document residual risks after control implementation

Scenario Planning With MITRE ATLAS

Adversarial Scenario Development Framework

Attack Chain Scenarios

Build complete attack narratives using ATLAS tactics

Scenario A: Model Extraction Attack

Reconnaissance: Attacker discovers public API documentation

Resource Development: Creates query generation infrastructure

Initial Access: Obtains legitimate API credentials

ML Attack Staging: Systematically queries model to extract parameters

Impact: Intellectual property theft, competitive disadvantage

Scenario B: Data Poisoning Campaign

Reconnaissance: Identifies crowdsourced training data collection

Resource Development: Creates network of fake accounts

Initial Access: Gains contributor access to data platform

Execution: Injects subtly poisoned samples over time

Impact: Model reliability degradation, targeted misclassifications

Scenario C: Adversarial Example Attack

Reconnaissance: Analyzes model behavior through API

Resource Development: Builds surrogate model for testing

ML Model Access: Crafts adversarial inputs

ML Attack Staging: Deploys examples in production

Impact: System bypass, incorrect decisions, reputation damage

Scenario Analysis Methodology

Analyze your scenarios with the following methodology:

- **Attack Path Mapping:** Visualize possible attack paths using ATLAS framework
- **Capability Requirements:** Assess technical skills and resources needed
- **Detection Opportunities:** Identify observable indicators at each stage
- **Impact Assessment:** Quantify potential business and technical impacts
- **Control Effectiveness:** Evaluate how current controls would perform

Authorized Security Testing Through Simulated Attack

- Select high-priority scenarios for testing
- Define rules of engagement and success criteria
- Execute attacks following ATLAS methodology
- Document techniques used and defenses encountered
- Measure blue team detection and response capabilities

Defensive Planning Based on Scenarios

Take a proactive approach to defensive planning following scenario analysis:

- **Playbook Development:** Create response procedures for each scenario type
- **Detection Rules:** Develop specific monitoring rules for ATLAS techniques
- **Training Programs:** Use scenarios for security awareness and incident response training
- **Control Validation:** Test defensive controls against scenario conditions
- **Metrics and KPIs:** Establish measurements for defensive readiness

Continuous Scenario Refinement

Utilize the approaches below to continuously refine your scenarios:

- Monitor ATLAS updates for new techniques and case studies
- Analyze internal incidents using ATLAS taxonomy
- Share scenarios with industry peers (sanitized)
- Update scenarios based on:
 - Emerging threat intelligence
 - New AI system deployments
 - Changes in threat landscape
 - Lessons learned from exercises

Integration With Security Operations

SIEM Integration

- Create detection rules based on ATLAS indicators
- Develop ML-specific security use cases
- Establish baselines for normal ML system behavior
- Configure alerts for anomalous patterns

Incident Response Procedures

- Adapt IR procedures for ML-specific incidents
- Define escalation criteria for ML attacks
- Establish evidence collection procedures
- Create remediation guidelines for each attack type

Threat Intelligence Sharing

- Use ATLAS taxonomy for consistent reporting
- Participate in ML security communities
- Share anonymized attack patterns
- Contribute to ATLAS case study database

5.12 Summary

Congratulations on completing Module 5! You now have comprehensive knowledge of the complete AI model development life cycle and the critical security challenges that emerge at every stage. You can systematically identify and analyze sophisticated AI-specific threats, apply industry-standard risk assessment frameworks including NIST AI RMF, ISO/IEC standards, and MITRE ATLAS, and implement robust threat modeling and continuous risk management practices to protect AI systems throughout their operational life cycle.

Key Concepts Mastered

AI Model Development Life Cycle Mastery

You've explored the critical phases of AI development: problem definition, data collection, preprocessing, exploratory data analysis, model selection, training, evaluation, deployment, monitoring, and documentation and governance. You learned how each phase creates unique risk profiles and vulnerability points, from ensuring AI is the appropriate solution during problem definition to implementing continuous monitoring for data drift and concept drift in production. You now understand how comprehensive documentation and governance transforms ML projects from experimental prototypes into professional, auditable systems.

AI-Specific Threat Analysis and Classification

You learned about identifying and analyzing sophisticated attack vectors unique to AI systems. You also learned about data poisoning attacks (including label flipping, backdoor attack, and clean-label poisoning), model manipulation techniques (backdoor injection, weight perturbation, and logic bombs), evasion attacks using adversarial examples, model extraction for intellectual property theft, and membership inference attacks that threaten privacy. You now understand how these threats operate differently from traditional cyber security threats and require specialized defensive strategies.

Risk Assessment Frameworks and Implementation

You investigated three major industry frameworks for AI risk management. With NIST AI RMF, you learned the four core functions (GOVERN, MAP, MEASURE, MANAGE) and how to establish organization-wide risk culture, define context and stakeholders, evaluate trustworthiness characteristics, and implement continuous monitoring. You explored ISO/IEC standards including the AI-specific risk management guidance (23894:2023) and the first certifiable AI management system standard (42001:2023). Finally, you gained proficiency with MITRE ATLAS as a means of understanding adversarial threat landscapes and developing comprehensive defensive strategies.

Threat Modeling and Scenario Planning

You learned to apply systematic threat modeling methodologies, particularly STRIDE adapted for AI systems, to proactively identify vulnerabilities across spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege. You can now conduct scenario-based risk analysis ranging from baseline operating conditions to black swan catastrophic events, and you understand how to

implement periodic risk review processes that ensure AI systems remain resilient as threats evolve and new vulnerabilities emerge.

Looking Ahead: Your Continued AI Journey

The security and life cycle knowledge you've developed in Module 5 provides the foundation for building AI systems that are not only efficient but secure, resilient, and trustworthy. You now understand that AI security requires a different approach than traditional cyber security, with threats that can emerge from data manipulation, algorithmic vulnerabilities, and the probabilistic nature of AI decision-making.

The frameworks and methodologies you've learned will become essential tools for every project. The NIST AI RMF will guide your governance and risk management processes, while MITRE ATLAS will help you anticipate and defend against evolving adversarial threats. Your understanding of the complete AI life cycle ensures you can identify and mitigate risks at every development stage, from initial data collection through long-term production monitoring.

AI security is an ongoing discipline that requires continuous vigilance and adaptation. New attack vectors emerge regularly, regulatory requirements continue to evolve, and AI systems themselves become more complex and capable. The systematic approaches you've learned for threat modeling, risk assessment, and continuous monitoring will enable you to build AI systems that can withstand both current and future challenges while maintaining the trust of users, stakeholders, and society at large.

Change Log

Version #	Approved by	Changes
20251015	HMR	Initial release